



PostgreSQL – Data Definition Language

Stephan Karrer

Datenbankobjekte

- Möglichst alle durch das DBMS verwalteten Einheiten werden als Datenbankobjekte präsentiert.
- Es gibt somit eine Vielzahl von Objekten die mittels der DDL-Anweisungen (CREATE, ALTER, DROP) erzeugt, verändert und gelöscht werden können:
 - Tabellen
 - Views
 - Indizes
 - Schemata/Datenbanken
 - Gespeicherte Prozeduren
 -

Erzeugen von Tabellen mit der CREATE-Anweisung

```
CREATE TABLE products (  
    product_no integer PRIMARY KEY,  
    name varchar(20) NOT NULL,  
    price numeric CHECK (price > 0),  
    discounted_price numeric,  
    CONSTRAINT dc CHECK (discounted_price > 0),  
    CHECK (price > discounted_price) -- uebergreifend  
);
```

- Constraints können benannt werden, ansonsten vergibt das System einen spezifischen Bezeichner
- Spaltenübergreifende und benannte Constraints müssen separat definiert werden (outline)

Constraints: Bedingungen auf Tabellen- bzw. Spalten-Ebene

Folgende Constraints sind in MariaDB zulässig (ANSI-konform):

- NOT NULL erlaubt keine NULL-Werte
- UNIQUE erlaubt nur eindeutige oder NULL-Werte
- PRIMARY KEY Primärschlüssel (NOT NULL und UNIQUE)
- FOREIGN KEY legt eine Fremdschlüsselbeziehung fest
- CHECK (C) gibt eine/mehrere Bedingung(en) an, die erfüllt sein müssen

Constraints können entweder beim Anlegen mit CREATE TABLE oder nachträglich über ALTER TABLE gesetzt werden.

Primär- und Fremdschlüssel festlegen

```
CREATE TABLE dept(deptno INT PRIMARY KEY,  
                   deptname VARCHAR(25)  
                   );  
  
CREATE TABLE emp (empno INT,  
                   ename varchar(10),  
                   deptno INT,  
                   CONSTRAINT pk_emp PRIMARY KEY(empno),  
                   CONSTRAINT fk_deptno FOREIGN KEY(deptno)  
                       REFERENCES dept(deptno)  
                   );
```

- Die referenzierte Tabelle und Spalte muss bereits existieren.

Referentielle Integrität: Ändern des Standardverhaltens

```
CREATE TABLE emp (empno INT,  
                   ename varchar(10),  
                   deptno INT,  
                   CONSTRAINT pk_emp PRIMARY KEY(empno),  
                   CONSTRAINT fk_deptno FOREIGN KEY(deptno)  
                       REFERENCES dept(deptno)  
                       ON DELETE CASCADE  
                       ON UPDATE SET NULL  
                   );
```

- Das Verhalten (default NO ACTION) kann getrennt für Update oder Delete des referenzierten Primärschlüssels festgelegt werden.

Automatische Schlüsselgenerierung

- Häufig sollen künstlich generierte eindeutige Werte für Schlüsselspalten verwendet werden
- 2 Verfahren sind üblich und auch ANSI-konform:
 - Auto-Increment: pro Spalte eigene Generierung von Ganzzahlen
 - Sequenzgenerator: eigenes Datenbank-Objekt, das übergreifend genutzt werden kann und Ganzzahlen produziert
- ANSI definiert dafür folgende Syntax:
"GENERATED { ALWAYS | BY DEFAULT } AS IDENTITY"
 - PostgreSQL hält sich an diese Syntax

Sequenzgenerator bei PostgreSQL

```
CREATE SEQUENCE seq1  
    INCREMENT BY 5  
    START WITH 10  
    MAXVALUE 10000  
    CACHE 50;
```

```
CREATE TABLE Test1 (  
    id bigint PRIMARY KEY DEFAULT nextval('seq1'),  
    name VARCHAR(99) );
```

-- oder automatischer Sequenzgenerator

```
CREATE TABLE Test2 (  
    id bigint GENERATED ALWAYS AS IDENTITY  
    (START WITH 100 INCREMENT BY 10));
```


Auto-Increment Spalte bei PostgreSQL

```
CREATE TABLE new_employees
(
  id_num integer PRIMARY KEY GENERATED BY DEFAULT AS IDENTITY,
  fname varchar (20),
  minit char(1),
  lname varchar(30)
);
```

- Die automatisch erzeugte Sequenz dient als Auto-Increment-Spalte

Tabellen durch Unterabfragen erstellen

```
CREATE TABLE dept_80 (  
    d80_emplid, d80_name, d80_jobid )  
AS SELECT employee_id,  
           first_name || last_name Name,  
           job_id  
FROM employees WHERE department_id = 80;
```

- Hersteller weichen manchmal von der Syntax ab, z.B. SQL SERVER

```
SELECT Customers.CustomerName, Orders.OrderID  
    INTO CustomersOrderBackup2017  
FROM Customers LEFT JOIN Orders  
    ON Customers.CustomerID = Orders.CustomerID;
```

ALTER TABLE: Spalten hinzufügen, ändern, löschen

```
ALTER TABLE countries
    ADD (duty_pct NUMERIC(2,2) CHECK (duty_pct < 10.5));
ALTER TABLE countries
    ADD COLUMN IF NOT EXISTS visa_needed VARCHAR(3);
```

```
ALTER TABLE countries
    ALTER COLUMN duty_pct TYPE DECIMAL(3,2);

ALTER TABLE countries
    ALTER COLUMN visa_needed SET DEFAULT 'no';
```

```
ALTER TABLE countries
    DROP COLUMN visa_needed;
```

TRUNCATE: Löschen aller Zeilen

```
TRUNCATE TABLE copy_emp;
```

- Entfernt alle Zeilen aus der Tabelle
- Ist effizienter als das Löschen aller Zeilen mit DELETE
- Tabellenstruktur verbleibt im Data Dictionary
- Es ist kein Rollback möglich

DROP: Löschen von Tabellen

```
DROP TABLE list_customers;
```

- Alle Daten und die Struktur der Tabelle werden gelöscht
- Alle Indizes für die Tabelle werden gelöscht
- Alle Constraints werden gelöscht
- Es ist kein Rollback möglich